

DMATH-ALGO

Mathematische Induktion

Herbstsemester 24

Serie 7

Lernziele

- Sie verstehen das Prinzip der mathematischen Induktion.
- Sie kennen den Unterschied zwischen schwacher und starker Induktion.
- Sie können Behauptungen induktiv beweisen.

Die mathematische Induktion ist eine Technik für Beweise, die direkt mit der Rekursion vergleichbar ist. Genauso wie eine rekursive Lösung eines Problems auf Lösungen für eine kleinere Instanz desselben Problems beruht, basiert ein induktiver Beweis einer Behauptung auf Beweisen für eine kleinere Instanz derselben Behauptung.

Ein umfassendes Verständnis der Rekursion hängt von einem gründlichen Verständnis der mathematischen Induktion ab. In der Informatik tauchen erstaunlich viele Anwendungen der mathematischen Induktion auf: Die Analyse der Korrektheit von Algorithmen, das Zählen der Anzahl von Bitstrings, die eine bestimmte Form haben, und viele andere.

Frage 1: Was ist mathematische Induktion?

Das Ziel der mathematischen Induktion ist es, für ein Prädikat $P(n)$ mit einer natürlichen Zahl n als Variable, zu beweisen, dass die Aussage

$$\forall n \in \{n \in \mathbb{N} : n \geq n_0\} : P(n)$$

wahr ist. Das Prädikat $P(n)$ ist demnach für alle natürlichen Zahlen n grösser einer Startzahl n_0 wahr.

Bei der *schwachen Induktion* gehen wir wie folgt vor:

- *Induktionsverankerung.*

Wir weisen nach, dass das Prädikat $P(n_0)$ für die Startzahl n_0 wahr ist:

$$P(n_0) \equiv \text{True}$$

- *Induktionsschritt.*

Wir weisen nach, dass für alle natürlichen Zahlen $n \geq n_0$ aus der Wahrheit von $P(n)$ die Wahrheit von $P(n+1)$ folgen muss:

$$\forall n \in \{n \in \mathbb{N} : n \geq n_0\} : (P(n) \Rightarrow P(n+1)) \equiv \text{True}$$

Die zweite Bedingung sorgt dafür, dass das Prädikat in der Tat für alle unendlich vielen natürlichen Zahlen $n \geq n_0$ wahr ist. Für $n_0 = 3$ gilt:

$$P(3) \quad \text{ist wahr} \quad (\text{Induktionsverankerung})$$

Weil $P(3) \Rightarrow P(4)$ wahr ist, muss auch $P(4)$ wahr sein

Weil $P(4) \Rightarrow P(5)$ wahr ist, muss auch $P(5)$ wahr sein

Weil $P(5) \Rightarrow P(6)$ wahr ist, muss auch $P(6)$ wahr sein

Weil $P(6) \Rightarrow P(7)$ wahr ist, muss auch $P(7)$ wahr sein

Weil $P(7) \Rightarrow P(8)$ wahr ist, muss auch $P(8)$ wahr sein

⋮

⋮

Mit diesem Verfahren können wir die **Korrektheit** von Algorithmen beweisen, deren Input mit natürlichen Zahlen parametrisiert ist.

Aufgabe 1: Es sollen vom Startwerte s die ersten n Zahlen, jeweils um d inkrementiert, zusammengezählt werden:

$$s + (s + 1d) + (s + 2d) + (s + 3d) + \dots + (s + (n - 1)d)$$

Ist der folgende Algorithmus `summe(s,n,d)` eine korrekte Implementierung für diese Funktion?

```
def summe(s,n,d):  
    return n*s + n*(n-1)/2*d
```

Verankerung:

$$\begin{aligned} & \text{summe}(s, 1, d) \\ &= 1 \cdot s + 1 \cdot \underbrace{(1-1)/2 \cdot d}_{=0} \\ &= s \end{aligned}$$

für $u=1$ ist der Algorithmus korrekt.

Schritt:

$$\begin{aligned} & \text{summe}(s, u, d) + s + u \cdot d \\ &= \underbrace{u \cdot s + u \cdot (u-1)/2 \cdot d}_{\text{für } u \text{ sei der Algorithmus korrekt,}} + s + u \cdot d \end{aligned}$$

$$= u \cdot s + s + \frac{u(u-1)}{2} \cdot d + u \cdot d$$

$$= (u+1) \cdot s + \left[\frac{u(u-1)}{2} + u \right] \cdot d$$

$$= (u+1) \cdot s + \left[\frac{u(u-1)}{2} + \frac{2u}{2} \right] \cdot d$$

$$= (u+1) \cdot s + \frac{u(u-1) + 2u}{2} \cdot d$$

$$= (u+1) \cdot s + \frac{u \cdot [(u-1) + 2]}{2} \cdot d$$

$$= (u+1) \cdot s + \frac{u \cdot (u+1)}{2} \cdot d$$

$$= \text{summe}(s, u+1, d)$$

dann ist der Algorithmus auch für $u+1$ korrekt.

Aufgabe 2: Es sollen vom Startwerte s die ersten n Zahlen, jeweils um q vervielfacht, zusammengezählt werden:

$$s + sq^1 + sq^2 + sq^3 + \dots + sq^{n-1}$$

Zeigen Sie die Korrektheit des folgenden Algorithmus mit schwacher Induktion.

```
def summe(s,n,q):
    return s*(q**n-1)/(q-1)
```

Verankerung:

$$\begin{aligned} & \text{summe}(s, 1, q) \\ &= s \cdot (q^1 - 1) / (q - 1) \\ &= s \cdot 1 \\ &= s \\ & \text{für } n=1 \text{ korrekt.} \end{aligned}$$

Schritt:

$$\begin{aligned} & \text{summe}(s, u, q) + s \cdot q^u \\ &= s \cdot \frac{q^u - 1}{q - 1} + s \cdot q^u \\ & \quad \text{für } u \text{ korrekt,} \\ &= s \cdot \left[\frac{q^u - 1}{q - 1} + q^u \right] = s \cdot \left[\frac{q^u - 1 + q^u(q - 1)}{q - 1} \right] \\ &= s \cdot \frac{q^{u+1} - 1}{q - 1} \\ &= \text{summe}(s, u+1, q) \text{ dann auch für } u+1. \end{aligned}$$

Aufgabe 3: Es sollen die n ersten Quadratzahlen zusammengezählt werden:

$$1^2 + 2^2 + 3^2 + 4^2 + \dots + n^2$$

Zeigen Sie die Korrektheit des folgenden Algorithmus mit schwacher Induktion.

```
def summQuad(n):
    return n*(n+1)*(2*n+1)/6
```

Verankerung:

$$\begin{aligned} & \text{summQuad}(1) \\ &= 1 \cdot (1+1) \cdot (2 \cdot 1 + 1) / 6 \\ &= 1 \cdot 2 \cdot 3 / 6 \\ &= 1^2 \\ & \text{für } n=1 \text{ korrekt.} \end{aligned}$$

Schritt:

$$\begin{aligned} & \text{summQuad}(u) + (u+1)^2 \\ &= \frac{u(u+1)(2u+1)}{6} + (u+1)^2 \\ & \quad \text{für } u \text{ korrekt,} \\ &= \frac{u(u+1)(2u+1)}{6} + \frac{6(u+1)^2}{6} \\ &= \frac{u(u+1)(2u+1) + 6(u+1)^2}{6} \\ &= \frac{(u+1)[u(2u+1) + 6(u+1)]}{6} \\ &= \frac{(u+1)[2u^2 + 7u + 6]}{6} = \frac{(u+1)(u+2) \cdot (2u+3)}{6} \\ &= \text{summQuad}(u+1) \\ & \quad \text{dann auch für } u+1. \end{aligned}$$

Aufgabe 4: Es sollen die ersten n ungeraden Zahlen zusammengezählt werden:

$$1 + 3 + 5 + 7 + \dots + (2n - 1)$$

Zeigen Sie die Korrektheit des folgenden Algorithmus mit schwacher Induktion.

```
def summe(s, n, q):  
    return n**2
```

Verankerung:

$$\begin{aligned} & \text{summe}(1) \\ &= 1^2 \\ &= 1 \\ & \text{für } n=1 \text{ korrekt.} \end{aligned}$$

Schritt:

$$\begin{aligned} & \text{summe}(n) + (2n+1) \\ &= \underbrace{n^2}_{\text{für } n \text{ korrekt,}} + (2n+1) \\ &= n^2 + 2n + 1 \\ &= (n+1)^2 \\ &= \text{summe}(n+1) \\ & \text{dann auch für } n+1. \end{aligned}$$

Aufgabe 5: Zeigen Sie mit schwacher Induktion, dass die folgende while-Schleife nicht terminiert.

```
n = 0  
while (8**n - 3**n)%5 == 0:  
    print('Hallo Welt!')  
    n = n + 1
```

Verankerung:

$$\begin{aligned} & (8^0 - 3^0) \% 5 == 0 \\ & (1 - 1) \% 5 == 0 \\ & 0 \% 5 == 0 \\ & \text{für } n=0 \text{ True.} \end{aligned}$$

Schritt:

$$\begin{aligned} & (8^{n+1} - 3^{n+1}) \% 5 == 0 \\ & (8^n \cdot 8 - 3^n \cdot 3) \% 5 == 0 \\ & [8^n(5+3) - 3^n \cdot 3] \% 5 == 0 \\ & [8^n \cdot 5 + 8^n \cdot 3 - 3^n \cdot 3] \% 5 == 0 \\ & [\underbrace{8^n \cdot 5}_{\text{durch 5 teilbar}} + \underbrace{(8^n - 3^n) \cdot 3}_{\substack{\text{durch 5} \\ \text{teilbar}}} \% 5 == 0 \\ & \text{für } n \text{ True,} \\ & \text{dann auch für } n+1. \end{aligned}$$

Aufgabe 6: Zeigen Sie mit schwacher Induktion, dass die folgende while-Schleife nicht terminiert.

```
n = 0
while 9**n%10 == 1 or 9**n%10 == 9:
    print('Hallo Welt!')
    n = n + 1
```

Verankerung:

$$9^0 \% 10 == 1 \text{ or } 9^0 \% 10 == 9$$
$$1 \% 10 == 1 \text{ or } 1 \% 10 == 9$$
$$\text{True or False}$$

für $n=0$ True.

Schritt:

$$9^{u+1} \% 10 == 1 \text{ or } 9^{u+1} \% 10 == 9$$
$$9 \cdot 9^u \% 10 == 1 \text{ or } 9 \cdot 9^u \% 10 == 9$$
$$\text{für } u \text{ True} \Rightarrow 9^u \% 10 = 1 \vee 9^u \% 10 = 9$$
$$\begin{array}{l} 9 \cdot 9^u \% 10 \\ = 9 \cdot 1 \% 10 \\ = 9 \end{array} \quad \begin{array}{l} 9 \cdot 9^u \% 10 \\ = 9 \cdot 9 \% 10 \\ = 1 \end{array}$$

dann auch für $u+1$.

Aufgabe 7: Zeigen Sie mit schwacher Induktion, dass die folgende while-Schleife nicht terminiert.

```
n = 0
while 2**n%7 == 1 or 2**n%7 == 2 or 2**n%7 == 4:
    print('Hallo Welt!')
    n = n + 1
```

Verankerung:

$$2^0 \% 7 == 1 \text{ or } 2^0 \% 7 == 2 \text{ or } 2^0 \% 7 == 4$$
$$1 \% 7 == 1 \text{ or } 1 \% 7 == 2 \text{ or } 1 \% 7 == 4$$
$$\text{True or False or False}$$

für $n=0$ True.

Schritt:

$$2^{u+1} \% 7 == 1 \text{ or } 2^{u+1} \% 7 == 2 \text{ or } 2^{u+1} \% 7 == 4$$
$$2 \cdot 2^u \% 7 == 1 \text{ or } 2 \cdot 2^u \% 7 == 2 \text{ or } 2 \cdot 2^u \% 7 == 4$$

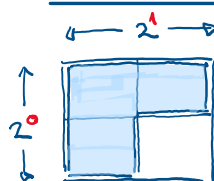
$$\text{für } u \text{ True} \Rightarrow 2^u \% 7 = 1 \vee 2^u \% 7 = 2 \vee 2^u \% 7 = 4$$

$$\begin{array}{l} 2 \cdot 2^u \% 7 \\ = 2 \cdot 1 \% 7 \\ = 2 \end{array} \quad \begin{array}{l} 2 \cdot 2^u \% 7 \\ = 2 \cdot 2 \% 7 \\ = 4 \end{array} \quad \begin{array}{l} 2 \cdot 2^u \% 7 \\ = 2 \cdot 4 \% 7 \\ = 1 \end{array}$$

dann auch für $u+1$.

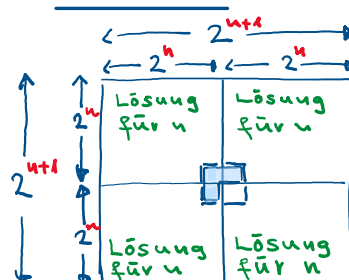
Aufgabe 8: Ein Algorithmus $\text{findeÜberdeckung}(n)$ soll eine Parkettierung mit Winkel-Fliesen (drei aneinanderhängende Quadrate, die nicht eine Linie bilden) auf einem Schachbrett mit 2^n Linien und 2^n Reihen finden. Alle Schachbrettfelder, mit Ausnahme von einem, werden genau einmal von einer Winkel-Fliese überdeckt. Zeigen Sie mit schwacher Induktion, dass eine solche Parkettierung immer möglich ist, dass also ein Algorithmus implementiert werden kann, der immer so eine Lösung findet.

Verankerung:



für $n=1$ gibt es eine Lösung.

Schritt:



dann auch für $n+1$.

Aufgabe 9 (optional): Ein Algorithmus $\text{findeWeg}(n)$ soll einen Weg für einen Springer auf einem Schachbrett mit n Linien und n Reihen finden, so dass der Springer jedes Feld mindestens einmal besucht. Zeigen Sie mit schwacher Induktion, dass so ein Weg für alle Schachbretter mit $n \geq 4$ immer existiert, dass also ein Algorithmus implementiert werden kann, der immer so einen Weg für einen Springer findet.

Frage 2: Was ist starke Induktion?

Im Unterschied zu der schwachen Induktion können wir bei der **starken Induktion** alle bereits gezeigten Prädikate $P(k)$ beim Induktionsschritt verwenden:

- **Induktionsverankerung.**

Wir weisen nach, dass das Prädikat $P(n_0)$ für die Startzahl n_0 wahr ist:

$$P(n_0) \equiv \text{True}$$

- **Induktionsschritt.**

Wir weisen nach, dass für alle natürlichen Zahlen $n \geq n_0$ aus der Wahrheit von $P(n_0), P(n_0 + 1), \dots, P(n)$ die Wahrheit von $P(n + 1)$ folgen muss:

$$\forall n \in \{n \in \mathbb{N} : n \geq n_0\} : (P(n_0) \wedge \dots \wedge P(n) \Rightarrow P(n + 1)) \equiv \text{True}$$

Aufgabe 10: Ein Algorithmus $\text{münzen}(n)$ soll für einen Geldbetrag von $n \geq 4$ Franken eine Anzahl a Zweiffränker und eine Anzahl b Fünffränker bestimmen, so dass die Summe all dieser Münzen n Franken beträgt. Zeigen Sie mit starker Induktion, dass so eine Menge von Münzen immer existiert, dass also ein Algorithmus implementiert werden kann, der immer Anzahlen $a \geq 0$ und $b \geq 0$ findet.

Verankerung:

$$4 = 2 \cdot 2 + 0 \cdot 5$$

für $n=4$ gibt es eine Lösung.

$$5 = 0 \cdot 2 + 1 \cdot 5$$

für $n=5$ gibt es eine Lösung.

Schritt:

$$n+1 = (n-1) + 2$$

Lösungen für $4, 5, \dots, \underline{n-1}, n$

$$= \underline{a \cdot 2 + b \cdot 5} + 2$$

$$= (a+1) \cdot 2 + b \cdot 5$$

dann auch eine Lösung für $n+1$

Aufgabe 11: Zeigen Sie mit starker Induktion die Korrektheit von folgendem Sortieralgorithmus:

```
import random as rd
def quickSort(A):
    if len(A) <= 1:
        return A
    pivot = rd.randint(0, len(A)-1)
    L = []
    R = []
    for i in range(len(A)):
        if i != pivot:
            if A[i] < A[pivot]:
                L.append(A[i])
            else:
                R.append(A[i])
    L = quickSort(L)
    R = quickSort(R)
    return L + [A[pivot]] + R
```

Setze $\text{len}(A) = n$

Verankerung:

$\text{quickSort}([])$
 $= []$

für $n=0$ korrekt

$\text{quickSort}([a_i])$
 $= [a_i]$

für $n=1$ korrekt

Schritt:

$\text{quickSort}([a_1, a_2, \dots, a_{n+1}])$
 $= L + [a] + R$

alle Elemente in L sind kleiner als a
alle Elemente in R sind grössergleich als a
 $\text{len}(L) \leq n$ und $\text{len}(R) \leq n$

für $0, 1, 2, \dots, n-1, n$ korrekt
(L und R sind also sortiert)
dann auch für $n+1$